Software Development Kit

NioLink
Protocol Manual

Version: 2.0.0
Date: November 30th, 2017

This document is protected by copyright. Do not copy or publish this document or parts of it without written permission of RGB Photonics GmbH.

Product specifications and descriptions in this manual are subject to change without notice. RGB Photonics GmbH will not be responsible for errors and omissions in this manual or for direct or incidental damages in connection with the use of this device or information.

© 2017 RGB Photonics GmbH
Donaupark 13
93309 Kelheim
Germany

Telephone: +49 9441 1750 33 – 0
Website: http://www.rgb-photonics.com
E-Mail: sales@rgb-photonics.com

Contents

1 Overview 1
2 NioLink Communication Protocol 2
2.1 Layer Abstraction 2
2.2 Message Layer 2
2.3 Interface Layer 5
3 Controlling a Spectrometer 16
3.1 Device Commands for Spectrometers 16
3.2 Taking Spectra 18
4 Technical Support 21

1 Overview

This manual explains the NioLink communication protocol, which can be used to control one of our lasers or spectrometers with USB Type-C connector from your own software. In addition to USB, these devices also offer different communication interfaces on the front-side auxiliary connector like SPI and UART[1].

If you have a device with Mini-USB connector instead, please refer to the separate ShortLink Protocol Manual for these devices.

Alternatively, if your operating system is Windows and using the .NET framework is fine for you, you may find the RgbDriverKit.dll library easier to use.

Our Spectrometers with USB Type-C connector feature a new powerful electronics board that implements all spectrum processing steps listed in chapter 3.2 in the device. So when you read out a spectrum from the device, you get a fully processed spectrum. These devices also allow complex application-specific evaluation algorithms that we may implement as a custom solution according to your requirements.

The general documentation for the NioLink protocol can be found in chapter 2. The specific commands for controlling a spectrometer are described in chapter 3. For a quick start to controlling your spectrometer using NioLink, please read the section in chapter 2.3 that corresponds to your interface and then the step-by-step guide in chapter 3.2.

[1] The UART interface is very similar to RS-232. The only difference is that it uses voltage levels of 0 V (low) and 3.3 V (high).

2 NioLink Communication Protocol

All our devices with a USB Type-C connector can be controlled with a new communication protocol called NioLink that was developed specifically with the requirements of measurement, control and test devices in mind.

This chapter contains a detailed documentation of NioLink. Alternatively, if you prefer a hands-on approach, you may take a look at the sample code in the SDK and come back later for answers to more specific questions.

The requirements for the development of this protocol were:

- Easy to understand and implement, following the Principle of Least Surprise.
- Same message protocol and command syntax for different interfaces.
- Communication can be ASCII or binary (automatically detected).

Therefore a single data stream is used for sending and another one for receiving data. Each command from the host computer is followed by a reply from the device. The device does not send unsolicited data.

Why a new protocol? We couldn't find any existing communication protocol for measurement and test devices that meets these requirements in a satisfactory manner. For example, while the SCPI protocol is open, flexible and widely-used, it is very complicated to implement (819 pages of specifications) and carries over legacies from ancient times when copper was cheaper than silicon.

Please note that this chapter explains how to control a device (slave) from a host computer (master). Some devices may also act as a master themselves to control other devices. In this case, different protocols may be used.

2.1 Layer Abstraction

NioLink achieves flexibility by separating the interface layer from the message layer:

The **interface layer** provides the transmission of a number of bytes over a physical interface (like USB, RS-232, SPI etc.) between a host and a device. Its implementation depends on the actual interface that is used.

The **message layer** parses and processes commands from the host to the device and sends replies back to the host. It sits on top of the interface layer and its implementation is independent from the actual interface.

2.2 Message Layer

The communication between host and device is half-duplex. It is always initiated by the host sending a command code and additional data (if required) to the device. The device

then replies with a return code and possibly additional data. After reading the reply, the host can send the next command.

NioLink supports ASCII as well as binary communication, but so far only binary mode is implemented. If the fourth byte sent from the host is an ASCII character (value 32 to 127), then the whole message to the device is assumed to be in ASCII mode and the reply will be sent accordingly.

Command Codes

In binary mode, each message to the device starts with a 32 bit command code of the form `0x0000XYZZ`. The upper 16 bit are reserved for future use.

This command codes follow a logical structure as described below. You do not necessarily need to understand this structure. Instead, you could just use the command codes as demonstrated in the sample code.

Upper Nibble of Byte 1 (“X”)

This value specifies the type of the message:

`0x0` = Command	(may be followed by a parameter, if required)
`0x1` = Parameter	(get/set parameters that may be changed by the user)
`0x2` = Device property	(get values that cannot be changed by the user)
`0x3` = Measured value	(get a measured value)
`0x4` = Bulk data	(get/set larger chunks of binary data. May be followed by index.)

Lower Nibble of Byte 1 (“Y”)

This value determines what to do:

`0x0` = GET	(gets the value, can be used with all message types except Command)
`0x1` = SET	(sets the value, can be used with Parameter and Bulk data)
`0x2` = MIN	(gets the minimum value, can be used with Parameter)
`0x3` = MAX	(gets the maximum value, can be used with Parameter)
`0x4` = DEF	(gets the default value, can be used with Parameter)
`0x8` = TYP	(gets the type of this value, can be used with all except Command)
`0x9` = NAM	(gets the name of this value, can be used with all except Command)
`0xA` = UNT	(gets the unit of the value, can be used with all except Command)
`0xF` = LEN	(gets the number of values available)

Byte 0 (“ZZ“)

This value specifies the index of the value or the bulk data or the command number. For example, if byte 1 is 0x20 (= Get device property), then byte 0 = 0x00 refers to the Device ID, byte 0 = 0x01 to the serial number and so on (see table below).

Return Codes

In binary mode, each reply from the device starts with a 32 bit return code of the form `0x0000XXYY`. The upper 16 bit are reserved for future use.

Byte 0 (“YY“)

This byte is used to report to the host if the last message was processed successfully or, if not, to provide a corresponding error code.

Code	Name	Comment
0x00	OK	The operation was completed successfully.
0x01	Unknown command code	
0x02	Invalid parameter	e.g. the parameter was out of range
0x03	Missing parameter	
0x04	Invalid operation	e.g. the current state of the device does not allow this operation
0x05	Function not supported	This device does not support this operation.
0x06	Passcode invalid	A passcode required for certain operations was not correct.
0x07	Communication error	Protocol violations or time-outs. If this error occurs, it is generally recommended to try again a few times.
0x08	Internal error	An error occurred that cannot be fixed by the user. In this case please call customer support.
0x09	Unknown bootloader cmd	The device is in bootloader mode and has received a command that is not a valid bootloader command.

Byte 1 (“XX“)

This byte can store a sub code that provides further details about an error. So far it is only used during firmware updates and with interface commands.

Data Types

Parameters, Device Properties and Measured Values are represented by different data types. All of these except strings are stored as 32 bit integers. You can determine the data type by sending a Type request. This is a command of the form 0x000018ZZ, 0x000028ZZ, 0x000038ZZ, where “ZZ” specifies the index.

Return values of TYPE requests:

0 = Signed 32 bit integer
1 = Unsigned 32 bit integer
2 = Float (single precision, IEEE 754 compliant)
3 = Version (4 bytes unsigned AA.BB.CC.DD)

4 = String (null-terminated, UTF-8)
5 = Boolean
6 = Hex number (similar to unsigned integer, also used for bit maks)
7 = Date

Generic Commands

There are a few generic commands and device properties that are supported by all devices:

Commands for all devices:

Binary/Index	ASCII	Name	Comment
`0x00`	INIT	Initialize	Start communication; if applicable wake up device
`0x01`	BYEB	Bye	Stop communication; if applicable send device to sleep
`0x02`	SRES	SystemReset	Perform a device reset
`0x03`	PRES	ParameterReset	
`0xFF`	FUPD	StartBootloader	Start firmware update of device (with passcode)

Device properties for all devices:

`0x00`	DVID	DeviceID	Same as USB VID/PID
`0x01`	SERN	SerialNo	As String
`0x02`	MANF	Manufacturer	
`0x03`	NAME	ModelName	
`0x04`	HVER	HardwareVersion	
`0x05`	SVER	SoftwareVersion	

2.3 Interface Layer

The tasks of the interface layer are:

- Transmission of byte data in half-duplex mode.
- Detection of the end of a message by the receiver.
- Ensuring data integrity (if desired) and handshake (if required).
- Addressing one of several devices on the same bus (if applicable).
- Reporting communication errors to the sender, if a message cannot be delivered.
- Informing the host when a reply from the device is ready to be read.

The length of a message is determined by the interface layer because it depends on the type of interface used. For example:

- *RS-232 in ASCII mode:* The length is determined by sending a CR+LF terminator at the end of the message. This makes it easy to send and receive commands using a serial terminal.
- *USB:* A partly filled USB packet marks the end of a message. If the last packet is completely filled, another "zero length packet" is sent afterwards (as described in the USB specifications).
- *SPI:* The actual message is prefixed by a short header that also includes the length.

Commands that are directed towards the interface layer and not passed on to the message layer are called "interface commands". They have unique command codes (`0x0000Fxxx`) that are distinct from other commands.

If a device has more than one interface, it initially listens on all of them. As soon as one of these interfaces is used by the host computer (by sending a message or enumeration), it is selected for all further communication.

The individual interfaces are described below.

USB

All devices provide USB communication in accordance with the USB 2.0 specification (High Speed with up to 480 Mbit/s). On the host computer the communication is usually handled by generic USB drivers such as WinUSB or libusb.

Devices by RGB Photonics use the USB Vendor ID (VID) 0x276E. The Product ID (PID) depends on the device model:

- Qmini: 0x0208
- Qred: 0x0209
- Qwave: 0x020A

USB transfers data in different channels (or pipes) called "endpoints" (EP). The NioLink protocol uses one endpoint for sending and one for receiving data:

- IN EP = 0x81 (read pipe)
- OUT EP = 0x01 (write pipe)

UART / RS-232

The Universal Asynchronous Receiver Transmitter (UART) is an asynchronous serial interface that needs no separate clock signal/line. The host's serial port needs to be configured to 8-N-1 which denotes the eight (8) data bits, no (N) parity bit, and one (1) stop bit.

The date rate is 57600 baud (= bits per second) by default and can be changed in the device parameters (see the user manual for details). The host and device must be configured to use the same baud rate before reading or writing data. This baud rate can

be configured by the user to any value. Changes are applied after the next device reset or by disconnecting and reconnecting the device.

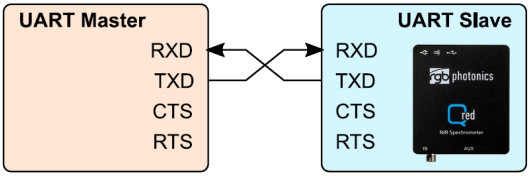
The standard baud rates are 300, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600, 115200, 128000, 230400, 256000, 460800, 921600 and 3000000. Any other value will be adjusted with the smallest error possible. Please note that a high baud rate might require flow control which is not yet supported.

Pin Description and Connection

The UART interface is available on the device's auxiliary connector. It uses two pins called "RXD" and "TXD" to communicate with the device. Optionally it can utilize the flow hardware control pins CTS and RTS (currently not supported yet). Please see the device user manual for a detailed pinout of the connector. A brief pin description is shown below:

Pin number	Signal name	Interface pin name	Pin description
6	UART_RXD	RXD	**Serial Input.** Serial receiver data
7	UART_TXD	TXD	**Serial Output.** Serial transmitter data
8	UART_CTS	CTS	**Clear To Send.** CTS is an asynchronous, active low modem status signal. An active low signal indicates if the external modem is ready to accept transmitted data via TXD from the UART.
9	UART_RTS	RTS	**Request To Send.** RTS is an active low signal informing the modem or data set that the UART is ready to receive data. An active low signal indicates that the UART would like to transmit data to the external modem.

The following diagram shows how to connect the device to a host in a simple configuration with no hardware flow control signals used:



The voltage levels for the UART interface are 0 V (low) and 3.3 V (high). The commonly used RS-232 serial interface is very similar, but uses a negative voltage for low and a positive voltage for high. If you would like to connect your device to a standard RS-232 serial port, please use our RS-232 converter cable (see user manual for details).

UART Interface Protocol

As described above, NioLink distinguishes between regular commands (directed towards the Message Layer) and interface commands (directed towards the Interface Layer).

The UART interface protocol uses interface commands to ensure the correct sequence of communication. It basically consists of a WRITE command to send byte data to the device. All interface commands to the device consist of a 32 bit command code followed by a 32 bit length information. All data is transferred with little-endian byte order.

The available interface commands for UART are:

Command Code	Description
`0x0000F102`	WRITE (to the device)
`0x0000F004`	GET_PROTOCOL_VERSION

Writing data ("payload") to the device is initiated by sending a WRITE command followed by the length of the payload and the actual payload data.

Because the UART interface is an asynchronous interface, reading from the device is initiated by the device itself. Therefore no extra READ command is necessary. The device responds with a 32-bit interface return code (see below) followed by the length of the data to be read out ("payload length").

WRITE to Slave + Length														
DIR	WRITE													
Block	Interface Command				Payload Length				Message Layer Data Write					
TXD	0x02	0xF1	0x00	0x00	Data(1)	Data(2)	Data(3)	Data(4)	Data(1)	Data(2)	Data(3)	Data(4)	...	Data(n)
RXD	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	...	0x00

READ from Slave														
DIR	READ													
Block	Interface Return Code				Payload Length				Message Layer Data Read (Message Return Code + Data)					
TXD	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	...	0x00
RXD	Data(1)	Data(2)	Data(3)	Data(4)	Data(1)	Data(2)	Data(3)	Data(4)	Data(1)	Data(2)	Data(3)	Data(4)	...	Data(n)

GET_INTERFACE_PROTOCOL_VERSION								
DIR	WRITE							
Block	Interface Command				Payload Length			
TXD	0x04	0xF0	0x00	0x00	0x00	0x00	0x00	0x00
RXD	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00

Response Time

READ							
Interface Return Code				Interface Protocol Version			
0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
Data(1)	Data(2)	Data(3)	Data(4)	Data(1)	Data(2)	Data(3)	Data(4)

Notes:

- Orange color indicates payload data to and from the message layer. All other colors stand for communication with the interface layer.
- The DIR row shows the direction during a frame, meaning whether the data is written to or read from the device.
- All data is transmitted in blocks of 4 bytes with little-endian byte order.
- The length of the payload is limited to 64 kB.

The Interface Return Codes consists of the 32 bit structure `0x0000XXYY` where the upper 16 bits are reserved for future use.

Byte 0 („YY“)

This byte is used to report the interface layer return code to the host.

Return Code	Name	Comments
0x00	OK	Command was processed successfully
0x01	UnknownCommandCode	Unknown Interface Command code
0x02	InvalidParameter	e.g. too few or too much bytes arrived
0x04	InvalidOperation	Miscellaneous but user-fixable error occurred
0x05	NotSupported	This device does not support this operation.
0x07	CommunicationError	Protocol violations or time-outs. Further details may be provided by a sub code in byte 1 (see below).
0x08	InternalError	Non user-fixable error occurred. Call support

Byte 1 („XX“)

This byte specifies interface-specific sub codes:

0x01 = BufferOverrun
0x02 = ChecksumError
0x03 = LengthError (inconsistent length information)

SPI

The Serial Peripheral Interface (SPI) is a synchronous serial bus with a master-slave setup used for short distance communication. Here the device acts as a slave with the host computer acting as the master that initiates all communication.

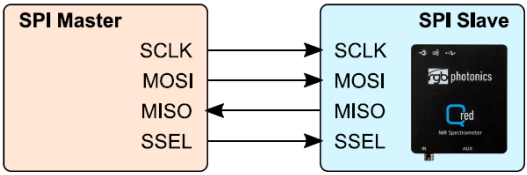
Pin Description and Connection

The SPI interface is available on the device’s auxiliary connector. It uses four wires to communicate with the device in slave mode. Please see the device user manual for a detailed pinout of the connector. A brief pin description is shown below:

Pin number	Signal name	Interface pin name	Pin description
11	SPI_MISO	MISO	The MISO signal transfers serial data from the slave to the master. When the Qseries device is a slave, serial data is output on this signal. When the slave is not selected by SSEL, it does not drive this signal (leaves it in high-impedance state).
12	SPI_CS	SSEL	The slave select signal qualifies the presence of data from the master, according to the protocol in use.

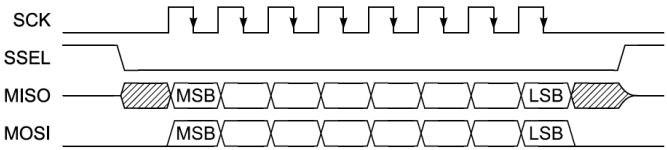
13	SPI_SCK	SCLK	SCLK is a clock signal used to synchronize the transfer data. It is driven by the master and received by the slave. When the SPI interface is used the clock is active-high, otherwise it is always high. SCLK only switches during a data transfer. Any other time, the SPI interface holds it in its inactive state, or does not drive it (leaves it in high-impedance state).
14	SPI_MOSI	MOSI	The MOSI signal transfers serial data from the master to the slave. When the Qseries device is a slave, it clocks in serial data from this signal.

The following diagram shows how to connect the device to a host in a simple configuration with no other devices:



Configuration and Functional Description

The device operated as a slave is configured with Clock Polarity (CPOL) = 0 and Clock Phase (CPHA) = 1, which is also known as SPI mode 1. An exemplary transfer sequence of 8 bits for this SPI frame format is shown below:



In this configuration, during idle periods:

- The SCLK signal is forced LOW.
- SSEL is forced HIGH (by the master).
- The transmit MOSI/MISO pins are high impedance.

The master starts a transmission by driving the SSEL signal LOW. This selects the slave. The master's MOSI pin is enabled. After that the SCLK is enabled with a rising edge transition. Data is then captured on the falling edges and propagated on the rising edges of the SCLK signal. After all bits have been transferred, the SSEL line is returned to its idle HIGH state one SCLK period after the last bit has been captured. For continuous back-to-

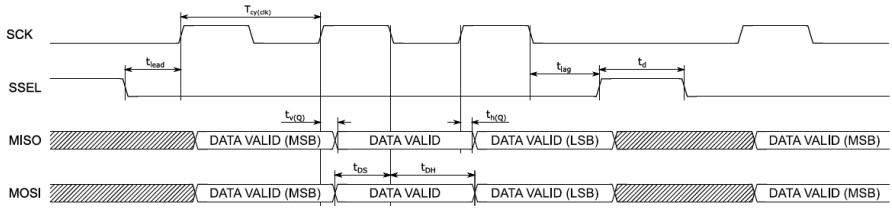
back transfers, the SSEL pin is held LOW between successive data words and termination is the same as mentioned above.

Dynamic Characteristics

The dynamic SPI characteristics (C_L = 20 pF, sampled at 10 % and 90 % of the signal level, CPOL=0, CPHA=1) are:

Symbol	Parameter	Min	Max	Unit
$T_{cy(clk)}$	clock cycle time	100	1000	ns
t_{DS}	data setup-time	1.5	-	ns
t_{DH}	data hold time	2	-	ns
$t_{v(Q)}$	data output valid time	-	23.22	ns
$t_{h(Q)}$	data output hold time	4.5	-	ns
t_{lead}	lead time	4000	-	ns
t_{lag}	lag time	0.5 x $T_{cy(clk)}$ + 1.5	-	ns
t_d	delay time	5000	-	ns

The SPI slave timing is:



SPI Interface Protocol

As described above, NioLink distinguishes between regular commands (directed towards the Message Layer) and interface commands (directed towards the Interface Layer).

The SPI interface protocol uses interface commands to ensure the correct sequence of communication. It basically consists of a WRITE command to send byte data to the device, a READ command to read byte data from the device and a command GET_STATUS to check if any data is available in the device to be read out. All interface commands to the device consist of a 32 bit command code followed by a 32 bit length information or 4 dummy bytes. All data is transferred with little-endian byte order.

The available interface commands for SPI are:

Command Code	Description

`0x0000F102`	WRITE (to the device)
`0x0000F000`	GET_STATUS
`0x0000F002`	READ (from the device)
`0x0000F004`	GET_PROTOCOL_VERSION

Writing data ("payload") to the device is initiated by sending a WRITE command followed by the length of the payload and the actual payload data.

Reading data from the device is started by sending a READ command to the device followed by four dummy bytes. The device then replies with a 32 bit interface return code (see below) followed by the length of the data to be read out ("payload length").

You can also use the GET_STATUS command to check if (and how much) data is available to be read from the device without actually reading it. Please note that in the current implementation it is mandatory to send a GET_STATUS command before issuing a READ command.

The diagram on the next page shows the detailed data flow for these interface commands:

WRITE to Slave + Length															
DIR	WRITE														
Block	Interface Command				Payload Length				Message Layer Data Write						
MOSI	0x02	0xF1	0x00	0x00	Data(1)	Data(2)	Data(3)	Data(4)	Data(1)	Data(2)	Data(3)	Data(4)	...	Data(n)	
MISO	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	...	0x00	

READ from Slave																							
DIR	WRITE								READ														
Block	Interface Command				Dummy				Interface Return Code				Payload Length				Message Layer Data Read (Message Return Code + Data)						
MOSI	0x02	0xF0	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	...	0x00	
MISO	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	Data(1)	Data(2)	Data(3)	Data(4)	Data(1)	Data(2)	Data(3)	Data(4)	Data(1)	Data(2)	Data(3)	Data(4)	...	Data(n)	

GET_STATUS																
DIR	WRITE								READ							
Block	Interface Command				Dummy				Interface Return Code				Payload Length			
MOSI	0x00	0xF0	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
MISO	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	Data(1)	Data(2)	Data(3)	Data(4)	Data(1)	Data(2)	Data(3)	Data(4)

GET_INTERFACE_PROTOCOL_VERSION																
DIR	WRITE								READ							
Block	Interface Command				Dummy				Interface Return Code				Interface Protocol Version			
MOSI	0x04	0xF0	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
MISO	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	Data(1)	Data(2)	Data(3)	Data(4)	Data(1)	Data(2)	Data(3)	Data(4)

Notes:

- Orange color indicates payload data to and from the message layer. All other colors stand for communication with the interface layer.
- The DIR row shows the direction during a frame, meaning whether the master is writing data to or reading from the slave.
- All data is transmitted in blocks of 4 bytes with little-endian byte order.
- The length of the payload is limited to 64 kB.

The Interface Return Codes consists of the 32 bit structure `0x0000XXYY` where the upper 16 bits are reserved for future use.

Byte 0 („YY“)

This byte is used to report the interface layer return code to the host.

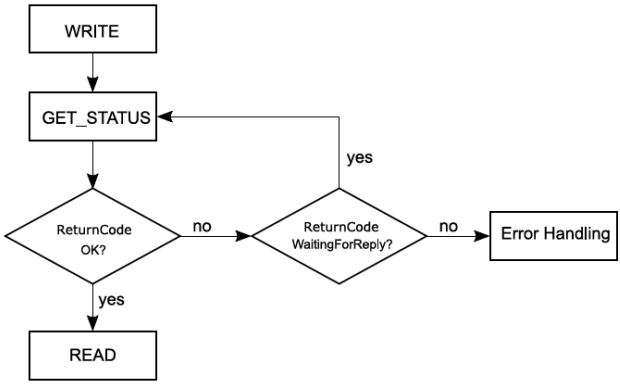
Return Code	Name	Comments
0x00	OK	- GET_STATUS: Data from Message Layer are now ready to be read - READ: Data from Message Layer are now arriving - All other commands: Command was processed successfully
0x01	UnknownCommandCode	Unknown Interface Command code
0x02	InvalidParameter	e.g. too few or too much bytes arrived
0x04	InvalidOperation	Miscellaneous but user-fixable error occurred
0x05	NotSupported	This device does not support this operation.
0x07	CommunicationError	Protocol violations or time-outs. Further details may be provided by a sub code in byte 1 (see below).
0x08	InternalError	Non user-fixable error occurred. Call support
0x0A	NoDataAvailable	There is no data to read and that won’t change unless you send a new command.
0x10	WaitingForReply	A message was received and is currently being processed.

Byte 1 („XX“)

This byte specifies the interface-specific sub codes:

0x01 = BufferOverrun
0x02 = ChecksumError
0x03 = LengthError (inconsistend length information)

A typical WRITE/READ sequence is illustrated in the following interface flow diagram, with the mandatory GET_STATUS command issued before actually reading data from the device:


WRITE
GET_STATUS
yes
ReturnCode
OK?
no
ReturnCode
WaitingForReply?
no
Error Handling
yes
READ

3 Controlling a Spectrometer

Spectra can be taken and transferred to the host by a series of simple commands and requests. The next section lists all command codes for spectrometers. A step-by-step guide on how to measure spectra can be found afterwards.

3.1 Device Commands for Spectrometers

Commands

Binary/Index	ASCII	Name	Comment
0x00	INIT	Initialize	Start communication; if applicable wake up device
0x01	BYEB	Bye	Stop communication; if applicable send device to sleep
0x02	SRES	SystemReset	Perform a device reset
0x03		ParameterReset	
0xFF	FUPD	StartBootloader	Start firmware update of device (with passcode)
0x04	STRT	StartExposure	With parameter: number of exposures
0x05	CNCL	CancelExposure	

Device Properties

0x00	DVID	DeviceID	Same as USB VID/PID
0x01	SERN	SerialNo	As String
0x02	MANF	Manufacturer	
0x03	NAME	ModelName	
0x04	HVER	HardwareVersion	
0x05	SVER	SoftwareVersion	
0x06	MAXV	SpectrumMaxValue	In ADC values, refers to raw spectrum
0x07	PIXC	PixelCount	Number of real pixels, after binning and cropping

There are many more device properties for spectrometers describing advanced technical details of how spectra are taken and processed. In most cases you do not need to worry about them and a detailed explanation would be beyond the scope of this manual.

Parameters

`0x00`	TEXP	ExposureTime	In seconds
`0x01`	AVRG	Averaging	
`0x02`	PROC	ProcessingSteps	Please see list below
`0x03`	IOCF	IOConfiguration	
`0x04`	TGCF	TriggerConfiguration	
`0x05`	TGDL	TriggerDelay	
`0x06`	TGEN	ExternalTriggerEnable	
`0x07`	BAUD	BaudRate	For the UART interface
`0x08`	NLED	TurnOffLeds	If your measurement needs total darkness

Measured values

`0x00`	STAT	Status	See below
`0x01`	TEMP	SensorTemperature	in degree Celcius
`0x02`	PORT	IOPortStatus	
`0x03`	TIME	SysTick	Time since start of spectrometer
`0x04`	RMNG	RemainingExposures	Mostly for debugging
`0x05`	BUFC	BufferCount	Mostly for debugging

Bulk Data

`0x00`	SPEC	Spectrum	Read only
`0x01`	WAVL	Wavelengths	Read only
`0x02`	CALD	CalibrationData	Read/Write, with Index
`0x03`	USRD	UserData	Read/Write, with Index

3.2 Taking Spectra

Setting Exposure Parameters

To measure a spectrum, you may follow some or all of these steps:

- Send a Reset command (command code 0x0000) to the device. As with all commands and requests, the device replies with a return code of 0x0000 if the command was completed successfully, or an error code if an error occurred.
- Set the exposure time using the command 0x1100 followed by the exposure time in microseconds.
- Set the averaging value (= adding up multiple exposures) using the command 0x1101 followed by the number of spectra to be averaged over.
- You may also set the processing steps using the command 0x1102, but in most cases the default value should be just fine.
- Start taking a spectrum by sending the StartExposure command (0x0004) followed by the number of spectra you would like to obtain (usually just 1).

Waiting for the exposure to finish

After sending the StartExposure command, you may check the Status value to see if the exposure has already finished. If you request the Status value with the command 0x3000, the device replies with the return code 0x0000 followed by the Status value.

Byte 0 of this value can be one of the following:

0 = Idle
1 = WaitingForTrigger
2 = TakingSpectrum
3 = WaitingForTemperature (Reserved for future use.)
-1 = NotReady (The device is currently in a state in which it cannot take spectra.)
-2 = Busy (The device is currently busy doing something else.)
-3 = Error (An error occurred while waiting for the exposure to be finished.)
-4 = Closed = -4 (The device is not initialized or the connection is closed.)

Byte 1 and 2 of the Status value give the number of spectra available to be read out from the device.

When you send the StartExposure command, the status changes from Idle to either WaitingForTrigger or TakingSpectrum. After all requested spectra have been taken, it changes back to Idle.

As soon as a spectrum becomes available to be read out to the host, the number on available spectra (Byte 1 and 2) is increased by 1.

Processing steps

In order to obtain the most precise spectrum, a number of processing steps are applied to the raw spectra that are read out from the image sensor. These processing steps depend on your hardware configuration and the calibration data stored in the device. They are usually applied automatically, but you can turn them on and off individually by setting the correponding bits in the ProcessingStep parameter.

- **1 = Adjust Offset**
  Adjust the offset using dark and offset pixels. Includes spectrum trimming and changing spectrum orientation, if applicable. May also sort the pixels in the order of ascending wavelengths.
- **2 = Correct Nonlinearity**
  Correct the non-linearity of the intensity axis.
- **4 = Remove Permanent Bad Pixels**
  Remove bad pixels.
- **8 = Subtract Dark**
  Obtain a dark spectrum for the current exposure time and subtract it from the measured data.
- **16 = Remove Temporary Bad Pixels**
  Remove single pixel spikes. Used only in special cases.
- **32 = Compensate Straylight**
  Estimate the amount of stray light and compensate it. Not used so far.
- **64 = Normalize Exposure Time**
  Normalize the spectrum to an exposure time of 1 second.
- **128 = Sensitivity Calibration**
  Use the sensitivity calibration to correct the spectral sensitivity.
- **256 = CorrectPRNU**
  Correct the photo response non-uniformity. This smoothens the sensitivity curve while keeping its overall shape.
- **512 = AdditionalFiltering**
  Apply additional filtering to compensate device-specific properties.
- **1024 = ScaleTo16BitRange**
  Scale values to a range of 0 – 65534 and clip larger values to 65535.

Reading a Spectrum

You can read out a spectrum to the host using the GetSpectrum command (0x4000). The reply from the device starts with the return code as usual, followed by a 48 byte header and then followed by the actual spectrum.

The spectrum header contains parameters of the acquired spectrum as follows:

- unsigned int ExposureTime (in microseconds)
- int Averaging
- unsigned int TimeStamp
- float LoadLevel

- float Temperature (in °C)
- uint16_t PixelCount (the number of pixels)
- uint16_t PixelFormat (specifies the spectrum number format, usually 32 bit float values)
- uint16_t ProcessingSteps (specifies which processing steps were applied)
- uint16_t IntensityUnit
- int SpectrumDropped; //später evt. auch noch andere Flags
- float SaturationValue (0.0 = darkness, 1.0 = image sensor is saturated)
- float OffsetAvg
- float DarkAvg
- float ReadoutNoise

4 Technical Support

You can contact our engineers by sending an e-mail to support@rgb-photonics.com.

Our sales department can be reached by e-mail, phone, fax or regular mail:

RGB Photonics GmbH
Donaupark 13
93309 Kelheim
Phone: +49 9441 175033 0
Fax: +49 9441 175033 92
E-Mail: sales@rgb-photonics.com

You can find additional resources on our website
`http://www.rgb-photonics.com`
by clicking on “Support” on the menu bar.